Develop Generative AI Apps in Azure

Build, optimize, and govern AI applications with Microsoft Foundry

Rick Beerendonk
rick@oblicum.com

glasspaper oblicum

What You'll Learn

- Explain how Foundry, a project, and a model deployment fit together
- Send a first request and continue a conversation over many turns
- Let a model call tools instead of only producing text
- Choose between prompting, retrieval, and fine-tuning
- Keep an AI application safe, observable, and inside its limits

Problem: A Model Alone Is Not an Application

A language model is a function: text in, text out. That is all it does.

- ❌ It has never seen your data
- ❌ It forgets everything after each call
- ❌ It cannot look anything up or take an action
- ❌ It has no identity, no budget, no audit trail

Need: a place to host models, connect them to your data and tools, and watch what they do.

That place is **Microsoft Foundry**. Everything in this presentation is one of those missing pieces.

The Big Picture

```
Your application
  |     "What is our refund policy?"
  ▼

 ┌──────────────────────────────────────────────────────────┐
 │  Foundry resource          (the Azure resource)          │
 │    └── Project             (what your code calls)        │
 │          ├── Model deployment   gpt-4o                   │
 │          ├── Tools              code, web, files         │
 │          └── Knowledge          your documents           │
 └──────────────────────────────────────────────────────────┘
  |     grounded answer + citations + token usage
  ▼
Your user
```

One resource holds projects. A project holds everything one application needs.

glasspaper oblicum

Decoding the Jargon

Term	In plain words
Model	The trained brain, e.g. `gpt-4o`. You cannot call it directly
Deployment	Your named, callable copy of a model, with its own quota and filters
Endpoint	The URL your code sends requests to
Inference	One call to the model — sending text and getting text back
Token	A chunk of text (~4 characters). You pay per token, in and out
Embedding	Text turned into a list of numbers so that similar text lands close by
Grounding	Putting real source text into the prompt so the answer is based on facts
Tool	Something the model may ask your app to run: code, search, your function
Agent	A model that loops — think, call a tool, look at the result, think again

Every later slide uses these words. Come back here whenever one stops making sense.

Life of One Request

Everything in this presentation happens somewhere in these seven steps.

```
1  User message arrives
       |
       ▼
2  Input content filter ────────► blocked? stop here
       |
       ▼
3  Your instructions + earlier turns are added
       |
       ▼
4  Model generates ◄──────────────────┐
       |                              │
       ▼                              │
5  Wants a tool? ──► run the tool ────┘
       | no
       ▼
6  Output content filter ──────► blocked? stop here
       |
       ▼
7  Answer + token usage returned
```

Steps 4 and 5 form a loop. That loop is the entire difference between a chat app and an agent.

Azure AI Foundry

What is Microsoft Foundry?

Developer platform for AI solutions:

Capability	What you get
Model catalog	1,900+ models from Microsoft, OpenAI, Meta, Mistral, Hugging Face, Anthropic
Model deployment	Deploy, version, and scale models
Foundry Tools	Language, Speech, Translator, Document Intelligence, Content Understanding
Agents	Build and host AI agents with tools and knowledge
Evaluations	Test quality and safety before deployment
Tracing	Observe agent and inference activity

Recommended for all but the simplest AI solutions

Foundry Resource and Projects

[EXAM]

Architecture:

- **Foundry resource** — top-level Azure resource; contains one or more projects
- **Project** — working space; one project is the default
- Projects contain: model deployments, agents, tools (incl. MCP), knowledge (Foundry IQ)

Endpoints:

Purpose	Endpoint format
Project (Foundry SDK)	`https://{resource-name}.services.ai.azure.com/api/projects/{project-name}`
Azure OpenAI (OpenAI SDK)	`https://{resource-name}.openai.azure.com/openai/v1`

Minimum administration to connect an app: the **endpoint URI plus a subscription key**. OAuth tokens, SAS tokens, and certificate-based managed identity all add credential setup to manage.

Plan the AI Solution

Start with the capability, then choose the service:

AI capability	Typical Azure solution
Generative AI and agents	Foundry models, prompts, tools, and agents
Natural language processing	Azure Language text analysis
Computer speech	Azure Speech, audio models, and Voice Live
Computer vision	Multimodal models and Content Understanding
Information extraction	Document Intelligence and Content Understanding

Planning questions:

- What input and output modalities are required?
- Does the solution need private or current data?
- Is a deterministic API preferable to an autonomous agent?
- Which identity, data residency, safety, and latency requirements apply?

Retrieval Practice — Lab 1

Pause here and complete Lab 1 before continuing. Use the matching solution only after attempting each question.

glasspaper oblicum

Foundry Tools

Prebuilt APIs and models for predictable, focused tasks:

Tool	Common use
Azure Language	Entity extraction, sentiment, summarization, and PII
Azure Speech	Speech-to-text, text-to-speech, and real-time speech
Azure Translator	Text translation and transliteration
Azure Document Intelligence	Fields, tables, and layout from documents
Azure Content Understanding	Multimodal extraction from documents, images, audio, and video

Use a Foundry Tool when:

- The task is narrow and well-defined
- Consistent output matters more than open-ended reasoning
- Cost and latency should be predictable

Developer Tools and SDKs

Tool	Best for
Microsoft Foundry portal	Explore models, test prompts, configure agents and tools
Visual Studio Code + Foundry extension	Code-first development, YAML, source control
Microsoft Foundry SDK	Automate projects, models, agents, evaluations, and connections
OpenAI SDK	Portable chat and Responses API application code
Azure service SDKs	Focused APIs such as Language, Speech, and Document Intelligence

Recommended workflow: prototype in the portal, export or write SDK code, then test the application with the same model and deployment configuration.

SDK Choice: Foundry vs OpenAI

Use Foundry SDK (`azure-ai-projects`)	Use OpenAI SDK
Agents, evaluations, tracing	Maximum OpenAI compatibility
Connections, project governance	Portability across providers
Foundry direct models (Phi, DeepSeek)	Simple chat inference

```
pip install azure-ai-projects azure-identity openai
```

glasspaper oblicum

AIProjectClient Setup

```python
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

project_client = AIProjectClient(
    credential=DefaultAzureCredential(),
    endpoint="https://{resource-name}.services.ai.azure.com/api/projects/{project-name}"
)

openai_client = project_client.get_openai_client(api_version="2024-10-21")
```

Note: `credential` is the first parameter, `endpoint` is second

glasspaper oblicum

OpenAI SDK Direct (Azure OpenAI endpoint)

```python
from openai import OpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default"
)

openai_client = OpenAI(
    base_url="https://{resource-name}.openai.azure.com/openai/v1/",
    api_key=token_provider
)
```

Use when: maximum OpenAI library compatibility is required

glasspaper oblicum

RBAC Roles for Inference

[EXAM]

Authentication proves *who* you are. RBAC decides *what you may do*. `DefaultAzureCredential` plus `az login` still returns **HTTP 403** without the right role.

Role	Grants
Cognitive Services OpenAI User	Call model inference — least privilege choice
Cognitive Services User	Read resource, list keys — no OpenAI inference
Cognitive Services Data Reader	Read data — no inference
Contributor	Full resource management — far too broad

Rule: for an app that only sends prompts, assign **Cognitive Services OpenAI User**.

glasspaper oblicum

Provisioning Resources

[EXAM]

Create a multi-service resource (one key and endpoint for sentiment, OCR, and future services):

Choice	Value
HTTP method	**`PUT`** — creates a named ARM resource (`.../accounts/CS1`)
Resource `kind`	**`CognitiveServices`** — the all-in-one multi-service kind
Single-service	`TextAnalytics`, `ComputerVision` — separate key and endpoint each

`POST` is for action endpoints; `PATCH` only updates an existing resource.

Customer-managed key (CMK) encryption:

```
az cognitiveservices account create --kind OpenAI \
  --encryption '{"keySource":"Microsoft.KeyVault","keyVaultProperties":{...}}'
```

`keySource` defaults to `Microsoft.CognitiveServices` (Microsoft-managed key). Set it to **`Microsoft.KeyVault`** for CMK.

Deploy an image model (DALL-E, gpt-image-1): use **Microsoft Foundry** and the **Azure CLI**. Microsoft Graph is for Microsoft 365 data, not model deployment.

Connections in Bicep

[EXAM]

A project **connection** stores where a dependency lives and how to authenticate to it, so applications and agents do not carry credentials.

```
resource kvConnection 'Microsoft.CognitiveServices/accounts/projects/connections@2025-04-01-preview' = {
  name: 'kv-connection'
  properties: {
    category: 'AzureKeyVault'          // must match the resource class
    target: existingKeyVault.id
    authType: 'AccountManagedIdentity' // identity-based, no stored secret
  }
}
```

Property	Why this value
`category`	Identifies the connected resource class — `AzureKeyVault`
`authType`	Key Vault authenticates by **identity**, not by an account key

Trap: an account-key or API-key `authType` puts a secret back into the connection — the opposite of the requirement.

Models

Catalog, deployment, benchmarks, and evaluation

glasspaper oblicum

Model Catalog

1,900+ models — category overview:

Category	Examples
LLMs	GPT-5, Mistral Large, Llama 3 70B
SLMs	Phi-4, Mistral OSS, Llama 3 8B
Reasoning	o-series (handles chain-of-thought internally)
Embedding	text-embedding-3-large
Image generation	gpt-image-1, FLUX
Video generation	Sora 2
TTS / STT	gpt-4o-tts, gpt-4o-transcribe
Multimodal	gpt-4.1, Phi-4-multimodal-instruct

Chat vs Reasoning: Non-reasoning models benefit from chain-of-thought prompting; o-series handle it internally — don't add "take a step-by-step approach" to o-series prompts

Model vs Deployment

You never call a model. You call **your deployment of** that model.

```
  Model in the catalog            Your deployment
  (shared, read-only)             (yours, named, configurable)

      gpt-4o           ⟶          "chat-prod"   quota, filters, version
      gpt-4o           ⟶          "chat-test"   smaller quota, looser filters
```

The deployment carries the things you control: **name, capacity, content filters, and version policy**. Two deployments of the same model can behave differently.

Why it matters: rate limits, cost, and safety settings are all per deployment — never per model.

Deployment Types

[EXAM]

Type	Description	Best for
Global Standard	Auto-routing, widest availability	Default choice for most workloads
Global Provisioned (PTU)	Dedicated throughput globally	Predictable, high-volume production
Global Batch	Async, 24-hour SLA, 50% cost	Large offline batch jobs
Data Zone Standard	Data residency within geographic zone	EU/regional data compliance
Data Zone Provisioned	Dedicated + data zone	High-volume + data compliance
Data Zone Batch	Async + data zone	Batch + data compliance
Standard	Single region, no rerouting	Single-region pinning

Use Global Standard whenever possible for maximum capabilities

Choosing a Deployment — Three Questions

[EXAM]

1. Where may the data be processed?

`Global` routes to any region with capacity — it **breaks an EU-only residency rule**. Use **Data Zone** for an EU/US boundary, or **Standard** (single region) to pin one region.

2. Is the demand steady or variable?

`Provisioned` reserves paid throughput and is wasted on variable or low-volume traffic. Variable interactive traffic → **Standard**.

3. Does it need a real-time answer?

`Batch` is asynchronous. Real-time REST → **Standard**, which is managed and does **not consume your VM vCPU quota** (a self-hosted container does).

4. Must behavior stay stable? Set the version-update policy to `OnceCurrentVersionExpired` so the version only changes at retirement.

Choosing a Model

[EXAM]

Requirement	Choose
Deep multi-step reasoning over retrieved documents	**LLM**
Simple, high-volume, cost-sensitive tasks	SLM
Vectors for search indexing and queries	Embedding model
Topics or terms only — no generated answer	Key phrase extraction

Model cascade — do not pick one model for everything:

```
  simple FAQ request  ➔  small model   cheap, fast
  complex reasoning   ➔  large model   accurate
```

A router classifies each request first. Sending everything to the smallest model loses quality; sending everything to the largest keeps cost and latency high; raising `max_tokens` for all requests only adds cost.

Model Benchmarks — Quality

Quality Index: 0–1 average across 8 benchmarks (higher = better)

Benchmark	Tests
Arena-Hard	Open-ended conversation quality
BIG-Bench Hard	Challenging reasoning tasks
GPQA	Graduate-level science Q&A
HumanEval+	Code generation
MBPP+	Python problem solving
MATH	Mathematical reasoning
MMLU-Pro	Multi-domain knowledge
IFEval	Instruction following

Model Benchmarks — Safety, Cost, Performance

Safety:

Benchmark	Metric	Better is
HarmBench	ASR (Attack Success Rate)	Lower
ToxiGen	F1 (toxicity detection)	Higher
WMDP	Hazardous knowledge accuracy	Lower

Cost: per 1M input tokens, per 1M output tokens, estimated cost (3:1 input:output ratio)

Performance: Latency mean, Latency P50, Throughput (TPM, tokens per second)

Evaluating a Model

[EXAM]

Evaluation targets: Model | Agent | Dataset

Dataset options: Upload new CSV/JSONL, Use existing, Generate synthetic (specify topic + row count)

Quality metrics (AI-assisted):

Metric	Description
Groundedness	Is every claim supported by the context?
Groundedness Pro	Binary: grounded or not
Relevance	Does the response answer the question?
Coherence	Is it logically structured?
Fluency	Is the language natural and correct?

Safety metrics: Defect rate = (true harmful instances / total) × 100
Evaluators: Self-harm, Hateful & unfair, Violent, Sexual, Protected material, Indirect attack

NLP metrics (no evaluator LLM needed): F1, BLEU, METEOR, ROUGE, GLEU

Chat Application

glasspaper oblicum

Responses API (Recommended for New Development)

[EXAM]

```
response = openai_client.responses.create(
    model="gpt-4o",
    input="What is the capital of France?",
    instructions="You are a helpful assistant.",
    temperature=0.7,
    max_output_tokens=200,
)

print(response.output_text)
print(response.id)           # for multi-turn chaining
print(response.status)
print(response.usage.total_tokens)
```

Works with: Azure OpenAI models AND Foundry direct models (Phi, DeepSeek, etc.)

Responses API — Multi-Turn

[EXAM]

Option 1: Chain by response ID (server manages history)

```
response2 = openai_client.responses.create(
    model="gpt-4o",
    input="What about Paris specifically?",
    previous_response_id=response1.id,
)
```

Option 2: Manual conversation history

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What is the capital of France?"},
    {"role": "assistant", "content": response1.output_text},
    {"role": "user", "content": "Tell me more."},
]
response2 = openai_client.responses.create(model="gpt-4o", input=messages)
```

Durable Conversation State

[EXAM]

Use a **conversation** when an agent must reuse complete interaction history across turns and sessions. A durable conversation ID links messages, tool calls, and tool outputs for the ongoing case.

Do not confuse: a response is one model result; an output item is part of a result; an agent is the reusable definition. The conversation is the durable runtime context.

Retrieval Practice — Lab 2

Pause here and complete Lab 2, then check its solution.

glasspaper oblicum

Persisting Conversation History

`previous_response_id` and manual history both reset when the session ends. For multi-session or multi-user apps, **persist the messages array externally**.

```
import json

# End of session — save
history = [
    {"role": "developer", "content": "System instructions..."},
    {"role": "user",      "content": user_input},
    {"role": "assistant", "content": response.output_text},
]
db.save(user_id, json.dumps(history))

# Start of next session — reload
history = json.loads(db.load(user_id))
history.append({"role": "user", "content": new_input})
response = openai_client.responses.create(model="gpt-4o", input=history)
history.append({"role": "assistant", "content": response.output_text})
db.save(user_id, json.dumps(history))
```

Storage options:

Streaming

```
with openai_client.responses.create(
    model="gpt-4o",
    input="Tell me a long story.",
    stream=True,
) as stream:
    for event in stream:
        print(event)
```

glasspaper oblicum

ChatCompletions API (Portability)

```
response = openai_client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is the capital of France?"},
    ]
)
print(response.choices[0].message.content)
```

Use when: maximum portability or OpenAI compatibility is required

	Responses API	ChatCompletions
Recommended for	New development	Portability
Multi-turn	`previous_response_id`	Manual history array
Result	`response.output_text`	`response.choices[0].message.cont`

glasspaper oblicum

Tools

Tools in the Responses API

[EXAM]

Specify tools in `responses.create()` — model decides when to use them:

Tool	Type name	Capability
Code interpreter	`code_interpreter`	Sandboxed Python runtime
Web search	`web_search_preview`	Live web retrieval
File search	`file_search`	Vector search over uploaded files
Function	`function`	Custom application logic

```
response = client.responses.create(
    model=DEPLOYMENT,
    input=[{"role": "user", "content": "What is the square root of 16?"}],
    tools=[{"type": "code_interpreter"}]
)
print(response.output_text)  # "The square root of 16 is 4."
```

Note: "Foundry Tools" (Language, Speech, etc.) are Azure AI APIs — distinct from these prompt tools

`code_interpreter` and `web_search`

[EXAM]

`code_interpreter` — model generates and runs Python in a sandbox:

- Pre-installed: `pandas`, `numpy`, `math`, `matplotlib`
- Can read/write files (CSV, JSON, images); auto-corrects errors
- No network access; timeout and memory limits apply
- Many models internally call this the *python tool*

`web_search` — model retrieves current information from the web:

- Use type name `"web_search_preview"` with Microsoft Foundry
- Also named **Grounding with Bing Search** `"bing_grounding"` in Foundry agents and exam wording
- Model decides when to search; issues query automatically
- Produces source-grounded answers for time-sensitive topics
- Adds latency and token usage; source quality varies

`file_search` and `function`

[EXAM]

`file_search` — semantic search over uploaded documents:

```
vector_store = client.vector_stores.create(name="policy-docs")
client.vector_stores.files.upload_and_poll(
    vector_store_id=vector_store.id, file=open("policy.pdf", "rb")
)
response = client.responses.create(
    model=DEPLOYMENT, input=messages,
    tools=[{"type": "file_search", "vector_store_ids": [vector_store.id]}]
)
```

`function` — your own code as a tool. The model never runs it; it asks you to.

Enterprise-scale multi-source knowledge: use Foundry IQ agents instead of `file_search`

glasspaper oblicum

Function Calling — Who Does What, When

The most common misunderstanding: the model does **not** execute your function.

```
Your app                                      Model
--------                                      -----
send: "Weather in Oslo?"          ──▶
+ tool schema: get_weather
                                  ◀──   "call get_weather(city='Oslo')"
                                                  (a request, not an answer)
run get_weather("Oslo")
= 4 degrees, rain

send the result back              ──▶
                                  ◀──   "It is 4 degrees and raining in Oslo."
```

Two round trips, always. The model chooses *whether* and *with which arguments*; your code stays in control of *what actually runs*.

Retrieval Practice — Lab 3

Pause here and complete Lab 3, then check its solution.

Optimization

Optimization Sequence

[EXAM]

Problem: Model lacks private/current data or doesn't respond in the right style

```
cheap, minutes        1  Prompt engineering   change what you *say*
      |               2  RAG                  change what it *knows*
costly, days          3  Fine-tuning          change what it *is*
```

Climb only one rung at a time. Most problems that look like "the model is wrong" are solved on rung 1 or 2.

Always establish a baseline before moving to the next step — otherwise you cannot tell whether the step helped.

Prompt Components and Patterns

[EXAM]

Components: System message, User message, Assistant message, Examples

Patterns:

Pattern	Description
Persona	"You are a seasoned marketing professional..."
Format template	Explicit output structure (bullets, JSON schema)
Chain-of-thought	"Take a step-by-step approach" (non-reasoning models only)
Few-shot	Input/output examples: `"Good" → "positive"`, `"Bad" →`

Use delimiters (`---`, Markdown headings, XML tags) to separate sections; watch for recency bias

Restricting scope (e.g. "answer only about Contoso products") belongs in the **system message**. Few-shot examples demonstrate a pattern; they do not enforce a boundary.

Model Parameters

[EXAM]

Temperature: higher = creative; lower = deterministic
Top P: limits vocabulary to top P probability mass

Adjust temperature OR top_p — not both

Parameter	Low value	High value
`temperature`	Deterministic, consistent	Creative, varied
`top_p`	Constrained vocabulary	Diverse vocabulary

Reasoning models are different: when extended reasoning (`thinking`) is enabled, `temperature` **must be** `1` or omitted — `0` or `2` fails validation with HTTP 400. Control effort with the reasoning-effort setting (`low` for simple summaries) instead.

Consistency vs completeness: lower `temperature` makes wording repeatable. It does **not** add missing content, and neither does raising `max_tokens` on its own.

Fixing Incomplete Output

[EXAM]

Problem: the response keeps omitting required clauses.

No parameter can add content that was never generated:

- ❌ Lower `temperature` — only makes wording repeatable
- ❌ Higher `temperature` — only adds randomness
- ❌ Lower `max_tokens` — truncates even more
- ❌ Switch model without measuring — unproven
- ❌ Block the failing response — detects the gap, does not fill it

Solution: a retry evaluation (reflection pass) in application logic, before returning

```
answer = generate(prompt)
review = evaluate(answer, required_clauses)
if review.missing:
    answer = generate(prompt, feedback=review.missing)
return answer
```

Raising `max_tokens` is a valid **supporting** fix — it permits a complete answer, but it does not produce one.

RAG — Retrieval Augmented Generation

Problem: Models lack private/current data → hallucinate

Pattern: Retrieve → Augment → Generate

- **Retrieve:** Search an index for relevant documents
- **Augment:** Add retrieved content to the prompt as context
- **Generate:** Model responds grounded in retrieved documents

How it works:

- **Embeddings:** convert text into numbers. Similar meanings produce nearby vectors.
- **Azure AI Search:** keyword search finds exact words; vector search finds similar meaning. **Hybrid search combines both — recommended for RAG.**

RAG in Slow Motion

The model is never retrained. You simply hand it the right page before it answers.

```
Ahead of time (once)
  documents  ➜  split into chunks  ➜  embed  ➜  store in index

At question time (every request)
1  user asks     "How many holiday days do I get?"
2  embed the question, search the index
3  get back 3 chunks of the real HR handbook
4  build the prompt:
       "Answer using ONLY this context: &lt;3 chunks&gt;
        Question: How many holiday days do I get?"
5  model answers from the context, with citations
```

Key insight: steps 1–3 are search, step 5 is the model. RAG quality is usually a *search* problem, not a model problem.

Two Types of RAG — Classic vs Agentic

Aspect	Classic RAG (single-shot)	Agentic RAG (Foundry IQ retrieval engine)
Retrieval calls	**One** search per user turn	**Many** — the agent decides when and what to search again
Query rewriting	None (uses the user question verbatim)	Agent rewrites, decomposes, and retries failed queries
Multi-source	Typically one index	Federated across knowledge sources, picks the right one per step
Cost / latency	Low — 1 search + 1 generate	Higher — multiple search/reason iterations
When to choose	FAQ, single document set, deterministic answer	Complex questions, multi-hop reasoning, mixed sources

"Generative RAG" / agentic retrieval is the same idea: the LLM is given retrieval as a *tool* (`file_search`, AI Search) and runs its own loop — search → read → decide → maybe search again — instead of a fixed retrieve-then-generate pipeline.

glasspaper oblicum

RAG — Groundedness Gate

[EXAM]

Problem: Retrieved documents may not contain the needed information → model fills the gap with training data → hallucination

Solution: Verify context is sufficient before generation

Option A — LLM gate (pre-generation):

```
check = openai_client.responses.create(
    model="gpt-4o",
    instructions="Answer only YES or NO.",
    input=f"Does this context contain enough information to answer: '{query}'?\n\nContext:\n{context}"
)
if "YES" in check.output_text.upper():
    answer = generate_answer(context, query)
else:
    answer = "I don't have that information in the company documents."
```

Option B — Groundedness evaluator (post-generation):

```
from azure.ai.evaluation import GroundednessEvaluator

score = evaluator(query=query, context=context, response=answer)
if score["groundedness"] < 3:
    return "I cannot answer reliably from the available documents."
```

Option A **blocks** generation; Option B generates first, then filters

Fine-Tuning

When to fine-tune: consistent style/tone, specific output format, reduce prompt length, distillation, enhance tool usage

Technique: LoRA (Low-Rank Adaptation) — updates a smaller subset of parameters; faster and cheaper than full retraining

Types:

Type	Description
SFT	Supervised Fine-Tuning — prompt/response pairs
RFT	Reinforcement Fine-Tuning — grader rewards correct responses
DPO	Direct Preference Optimization — preferred vs. non-preferred pairs (lighter than RL)

Can combine methods (e.g., SFT first, then DPO for alignment)

Fine-Tuning — Training Data Format

JSONL format — one example per line:

```
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "What is the capital of France?"}, {"role": "assistant", "content": "Paris."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "What is 2 + 2?"}, {"role": "assistant", "content": "4."}]}
```

Always establish a baseline before fine-tuning

glasspaper oblicum

Responsible AI

glasspaper oblicum

The Six Responsible AI Principles

[EXAM]

Principle	Question it answers
Fairness	Are all groups treated equitably?
Reliability & safety	Does it behave dependably, including under stress?
Privacy & security	Is user data protected?
Inclusiveness	Can people of all abilities use it?
Transparency	Do users understand how it works and how their data is used?
Accountability	Is a named person answerable for the outcome?

Trap: "tell users how their data is processed" → **Transparency**. Not fairness (equitable treatment), inclusiveness (accessibility), or reliability and safety (dependable operation).

The 4-Stage Process

Map → Measure → Mitigate → Manage

Stage	Goal
Map	Identify and prioritize potential harms
Measure	Quantify the presence of harms in your system
Mitigate	Reduce harms at multiple layers
Manage	Deploy and operate responsibly

glasspaper oblicum

Retrieval Practice — Lab 4

Pause here and complete Lab 4, then check its solution.

Map Stage

1. **Identify harms** — by scenario, user type, potential misuse
2. **Prioritize** — by likelihood × impact
3. **Test/verify** — red team: deliberately probe for failures
4. **Document and share** — record findings for ongoing reference

Measure Stage

1. **Prepare** adversarial measurement prompts
2. **Submit** prompts to the model
3. **Apply evaluation criteria** — manual review first, then automate

Start manual, then automate — automation requires validated criteria

glasspaper oblicum

Mitigate Stage — 4 Layers

No single layer stops everything. They are checkpoints a request passes through.

```
    user input
      |
  [4] user experience    warnings, input limits, "AI can be wrong"
      |
  [3] system message     rules + grounding data in the prompt
      |
  [2] safety system      content filters, prompt shields
      |
  [1] model              a safer base model, or fine-tuned
      |
    answer  →  filtered again on the way out
```

Layer	Description
1. Model layer	Select an appropriate model; fine-tune for safer behavior
2. Safety system	Content filters (input/output) + prompt shields + protected material
3. System message & grounding	Behavioral parameters + RAG grounding
4. User experience	UI controls, input validation, transparency

Configure Content Filters

[EXAM]

Content filters are configured per deployment via **Guardrails + controls** in the Foundry portal. The configuration wizard has **4 pages.**

Pages 1 & 2 — Input filters and Output filters

Applied to user prompts (input) and model completions (output) separately:

Category	Configurable threshold (block at or above)
Hate & fairness	Low / **Medium** / High
Sexual	Low / **Medium** / High
Violence	Low / **Medium** / High
Self-harm	Low / **Medium** / High

Default: block Medium and above — content at Low or Safe passes through

glasspaper oblicum

Prompt Shields and Protected Material

[EXAM]

Page 3 — Prompt Shields (input)

Type	Default	Purpose
Direct attacks	On	Detect jailbreak attempts targeting the system prompt
Indirect attacks	Off	Detect malicious instructions embedded in **documents or images** the model reads

Indirect attack = Cross-Domain Prompt Injection: an attacker hides instructions inside content your agent processes (e.g., text in an image, a malicious PDF). Enable this when agents use RAG, File Search, or vision inputs.

Page 4 — Protected material (output)

Type	Default	Purpose
Text	On	Block known copyrighted content (lyrics, recipes)
Code	On	Block/cite public code (GitHub Copilot–powered)

glasspaper oblicum

Where to Moderate

[EXAM]

An agent has **four** content checkpoints, not two:

```
user input —[1]→ model —[2]→ tool call → tool —[3]→ tool response → model —[4]→ output
```

Moderating only user input and final output leaves the **tool call** and **tool response** unchecked.

Input surface	What actually protects it
Direct user message	**User prompt shield**
Text extracted from a document	**Document prompt shield** (indirect attacks)
Uploaded image	Image-specific inspection — user and document shields do **not** cover it

Traps: image moderation detects harmful *visuals*, not hidden instructions. Protected-material detection flags copyrighted content, not injection.

glasspaper oblicum

Thresholds and Spotlighting

[EXAM]

Threshold direction is counter-intuitive:

```
Low      blocks Low + Medium + High    ← most restrictive, most rejections
Medium   blocks Medium + High          ← default
High     blocks High only              ← least restrictive
```

Lowering a filter to **Low** *increases* rejections — it never loosens anything. **Protected material** is a text match, so it ignores severity thresholds entirely and its result is the same at every threshold.

Spotlighting marks external content (OCR text, retrieved documents) as untrusted **data**, so the model does not follow instructions found inside it.

Requirement	Setting
Stop detected prompt attacks	Prompt shields → **Block**
Lower the model's trust in external text	Enable **spotlighting**

Both are needed: shields without spotlighting do not lower trust; spotlighting without Block does not stop the attack.

Manage Stage

Pre-release reviews: Legal, Privacy, Security, Accessibility

Release and operate:

- **Phased delivery plan** — release to restricted group first; gather feedback
- **Incident response plan** — estimate time to respond to unanticipated incidents
- **Rollback plan** — steps to revert to previous state
- **Block capability** — immediately block harmful responses when discovered
- **User feedback** — allow reporting of inaccurate, harmful, or offensive content
- **Telemetry** — track user satisfaction; comply with privacy laws

Rate Limits and 429 Errors

[EXAM]

Units: TPM (Tokens Per Minute) and RPM (Requests Per Minute) — per deployment

Why you get 429 even when usage looks low:

- `max_tokens` is counted in the rate limit estimate upfront, not actual tokens generated
- RPM window is 1–10 seconds — a burst triggers it even if per-minute total is fine

Response headers (always present):

Header	When present	Meaning
`x-ratelimit-remaining-requests`	Every response	Remaining RPM budget
`x-ratelimit-remaining-tokens`	Every response	Remaining TPM budget
`retry-after-ms`	429 only	Wait time in milliseconds

SDK built-in retry: `AzureOpenAI(max_retries=5)` — default is 2; exponential backoff + `retry-after` header

Retry strategy: exponential backoff with jitter. Immediate retries hit the same limit; identical backoff across clients re-synchronizes them into a new burst.

HTTP 400 — request too large: move the bulk content into files and retrieve it with `file_search` instead of inlining it in the prompt.

Best practices:

- Set `max_tokens` to the **minimum that still produces complete answers** — over-setting wastes rate limit budget without improving quality (model stops naturally when done)
- Spread requests evenly — avoid sending all requests in the same 1-second window
- Distribute load across multiple deployments or regions
- Use **Provisioned (PTU)** deployments for stress testing — reserved throughput, no token-based rate limits

Retrieval Practice — Lab 5

Pause here and complete Lab 5, then check its solution.

glasspaper oblicum

Model Lifecycle and Retirement

[EXAM]

5 stages: Preview → GA → Legacy → Deprecated → **Retired**

Stage	Description
Preview	Early access; may change; 30-day retirement notice
GA	Generally available; 18-month lifecycle
Legacy	Replaced by newer version; still accessible
Deprecated	Blocked for new customers; existing customers can continue
Retired	**HTTP 410 Gone** — calls fail completely

Timeline (GA): At month 12 → Deprecated; at month 18 → Retired
Notifications: 60 days (GA), 30 days (Preview) — email to subscription owners + Azure Service Health (filter: "Azure OpenAI Service")

Auto-upgrade: Standard deployments only — Provisioned must migrate manually

`versionUpgradeOption` **property:**

Value	Behavior
`OnceCurrentVersionExpired`	Upgrade only when current version retires (safest)
`OnceNewDefaultVersionAvailable`	Upgrade when any new default version is released
`NoAutoUpgrade`	Never auto-upgrade — deployment dies at retirement

API terminology trap: `lifecycleStatus: "Deprecated"` in REST API = **Retired** (410); `"Deprecating"` = Deprecated (still works)

glasspaper oblicum

Monitor Your AI Application

[EXAM]

Azure Monitor collects metrics and logs automatically for all AI resources.

Two data paths:

Data	Collection	Where stored	Action needed
Platform metrics	Automatic	Azure Monitor metrics DB	None
Resource logs	Not automatic	Requires diagnostic setting	Create setting

Dashboards (Foundry portal → Metrics): HTTP Requests, Tokens-Based Usage, PTU Utilization, Fine-tuning

Enable resource logs:

1. Azure portal → your AI resource → **Diagnostic settings** → **Add**
2. Select log categories → route to **Log Analytics workspace**

Query logs with KQL:

```
AzureDiagnostics | take 100
| project TimeGenerated, OperationName, DurationMs, ResultSignature

AzureMetrics | take 100
| project TimeGenerated, MetricName, Total, Average, UnitName
```

Alerts: metric alerts (token usage thresholds), log alerts (KQL-based), activity log alerts (resource changes)

Application Insights: use for application-layer tracing, custom events, request/dependency tracking

Retrieval Practice — Lab 6

Pause here and complete Lab 6, then check its solution.

Which Signal Answers Which Question

[EXAM]

Symptom	Signal to read
Service-side outage	**Model Availability Rate**
HTTP 429 under load	Model Availability Rate **and Provisioned Utilization** (capacity)
Unexplained cost spike	**Token usage** — input and output size drives the bill
Low-quality or wrong answers	`RequestResponse` diagnostic logs
Ordered calls inside one run	**Tracing**

Diagnostic log categories:

Category	Contains
`RequestResponse`	Full request payloads, status codes, and completions
`Audit`	Access events only
`Trace`	Internal service operations only

Traps: latency measures time, run success rate measures completion, evaluation metrics measure quality — none of them explain cost.

Tracing Configuration

[EXAM]

Instrument with **OpenTelemetry**, collect and analyze in **Application Insights**. A Log Analytics workspace stores logs but does not instrument the app.

Requirement	Setting
Keep each service separate in the trace view	A distinct `OTEL_SERVICE_NAME` per service
Keep prompts and completions out of spans	`enable_content_recording=False`
Audit nested operations	**Hierarchical spans** — not flat log entries
Audit tool invocations	**Tool-call attributes** — not generic messages

For an agent: enable **Application Insights for the agent** in the Foundry project. Updating the agent version, restarting it, or attaching a Log Analytics workspace alone sends no telemetry.

Security Posture — Defender for Cloud and Sentinel

Azure Monitor / App Insights → *performance & cost* signals (latency, tokens, errors)
Microsoft Defender for Cloud + Microsoft Sentinel → *security & threat* signals for the AI workload

Service	Role for AI workloads
Defender for Cloud — AI workloads	Threat detection on Azure OpenAI / Foundry: prompt-injection alerts, data-leak detection, anomalous access on the resource
Defender for Cloud — secure score	Continuous configuration assessment: keyless auth, private endpoints, diagnostic settings on
Microsoft Sentinel (SIEM/SOAR)	Aggregates Defender alerts + diagnostic logs across resources; correlation rules; automated playbooks for incident response

Wiring it up:

1. Enable **Diagnostic settings** on the AI resource → Log Analytics workspace
2. Onboard the workspace to **Microsoft Sentinel**
3. Enable the **Defender for Cloud** AI workloads plan on the subscription
4. Sentinel ingests Defender alerts + `AzureDiagnostics` logs → unified detection across model abuse, key misuse, exfiltration

Exam framing: “health of the system or the data” in a security sense → Defender for Cloud (resource posture) and Sentinel (SIEM correlation), not Azure Monitor.

Operate and Secure AI Solutions

The production concerns that complete a generative AI application.

Identity, Network, Capacity, Delivery, Oversight

Production AI must answer five questions:

- **Who:** Entra ID, managed identity, and RBAC
- **From where:** private endpoints and network rules
- **How much:** RPM, TPM, PTU, concurrency, and token cost
- **How does change ship:** source control, evaluation, promotion, and rollback
- **What happened:** metrics, traces, evaluation, provenance, and audit

Keyless authentication still requires the correct role assignment. Private networking controls reachability, not authorization.

Production Observability and Governance

Resource logs require a diagnostic setting before they reach Log Analytics. Monitor application, model, and AI-quality signals together:

- Requests, dependencies, failures, latency, and user feedback
- Tokens, throughput, model versions, rate limits, and retries
- Groundedness, relevance, fluency, safety, and refusal rates

Use phased delivery, approval controls, provenance, incident response, and rollback for accountable operation.

Key Takeaways

1. **Foundry resource → Projects** (not Hub); use project endpoint for Foundry SDK, Azure OpenAI endpoint for OpenAI SDK
2. **1,900+ models** — 9 deployment types; use Global Standard by default
3. **Responses API** recommended for new development; ChatCompletions for portability
4. **Tools:** `code_interpreter`, `web_search_preview`, `file_search`, `function` — model decides when to call
5. **Optimization sequence:** Prompt engineering → RAG → Fine-tuning
6. **Fine-tuning techniques:** LoRA; types: SFT, RFT, DPO
7. **Responsible AI:** six principles (Transparency answers "how is my data used?"); Map → Measure → Mitigate (4 layers) → Manage
8. **Moderate four points:** user input, output, tool call, tool response; **Low threshold = most restrictive**; spotlighting + Block for injected instructions
9. **Incomplete output:** fix with a retry/reflection evaluation, not with temperature
10. **Rate limits:** TPM + RPM per deployment; retry with **exponential backoff + jitter**; set `max_tokens` to minimum needed
11. **Model retirement:** Retired = **HTTP 410**; Standard auto-upgrades; Provisioned must migrate manually
12. **Monitor:** platform metrics auto-collected; `RequestResponse` logs need a diagnostic setting; Provisioned Utilization explains 429
13. **Tracing:** OpenTelemetry + Application Insights; distinct `OTEL_SERVICE_NAME` per service; `enable_content_recording=False`
14. **RAG groundedness gate:** verify retrieved context is sufficient before generating — LLM gate (pre) or `GroundednessEvaluator` (post)
15. **Cross-session history:** `previous_response_id` resets per session — persist messages array externally (Cosmos DB, Redis)

Problem: It Worked in the Playground

The demo was approved on Friday. On Monday it is real software.

- ❌ The key is in a config file that three teams can read
- ❌ Anyone on the internet can reach the endpoint
- ❌ At 09:00 every request returns `429 Too Many Requests`
- ❌ A prompt was changed and nobody knows by whom, or when
- ❌ A customer asks why the model said that — and there is no record

Need: the same discipline you already apply to any production system, translated to AI.

Five Questions Every Production AI System Must Answer

```
  1  WHO can call it?           identity    Entra ID, managed identity, RBAC
  2  FROM WHERE?                network     private endpoint, network rules
  3  HOW MUCH can it use?       capacity    RPM, TPM, PTU, token cost
  4  HOW does a change ship?    delivery    source control, pipeline, rollback
  5  WHAT actually happened?    oversight   metrics, traces, evaluation, audit
```

The rest of this presentation is one section per question.

Exam habit: when a question describes a production failure, first decide *which of the five* it belongs to. That usually eliminates three of the four answers.

Identity: Keys, Tokens, and Managed Identity

[EXAM]

Key-based authentication: simple to start with, but secrets must be stored and rotated securely.

Microsoft Entra ID: preferred for production applications.

Managed identity: Azure manages the application identity; no client secret is stored in code.

Exam decision pattern:

Scenario	Preferred approach
Local development	`DefaultAzureCredential` with developer login
Azure-hosted application	System-assigned or user-assigned managed identity
Temporary external client	Short-lived token with narrowly scoped permissions
Legacy or isolated integration	Key stored in Key Vault and rotated

Keyless does not mean permissionless: assign the identity only the required Azure roles.

Least-privilege role	Grants
Cognitive Services OpenAI User	Call model inference
Key Vault Secrets User	Read secrets — not Administrator
Storage Blob Data Reader	Read blobs — not Contributor (write/delete)

glasspaper oblicum

How Keyless Actually Works

A key is something you *have*. An identity is something you *are* — and Azure can vouch for it.

```
  With a key                          With managed identity

  app ─── key in config               app  ───(1) "who am I?"──▶ Azure platform
      \                                    ◀──(2) short-lived token ───
       └──▶ AI resource                    ───(3) call with token ─▶ AI resource
                                                                          |
  key leaks = permanent problem                    (4) checks RBAC role
                                                   token expires in minutes
```

`DefaultAzureCredential` hides this. Locally it uses your developer login; in Azure it uses the managed identity — same code, no secret either way.

Keyless does not mean permissionless: without an RBAC role assignment, step 4 fails with 403.

glasspaper oblicum

Private Networking and Data Protection

[EXAM]

Private networking controls how traffic reaches the AI resource.

- Private endpoint: expose the resource through a private IP in a virtual network
- Public network access: disable when all clients should use private connectivity
- Network rules: restrict allowed subnets and trusted services
- Egress controls: limit where retrieved documents and tool calls can go
- Data residency: select an appropriate region or data zone deployment

Exam distinction: authentication answers *who may call*; networking answers *from where the resource may be reached*.

Also consider: encryption, Key Vault, diagnostic settings, retention, redaction, and tenant isolation.

Retrieval Practice — Lab 7

Pause here and complete Lab 7, then check its solution.

Quotas, Scaling, and Cost

[EXAM]

Capacity is not one number. Three separate limits can each stop you.

```
RPM  requests per minute   how OFTEN you may call      burst of calls ➜ 429
TPM  tokens per minute     how MUCH text per minute    long prompts   ➜ 429
PTU  provisioned units     capacity you RESERVED       no token limit, fixed bill
```

A token is roughly four characters. A 2-page prompt is ~1,000 tokens, so 60 such calls a minute costs 60,000 TPM — even though it is only 60 requests.

Trap: your requested `max_tokens` is reserved up front. Asking for 4,000 output tokens spends 4,000 of your TPM budget even when the model replies with one word.

Capacity Dimensions and Levers

Capacity has multiple dimensions:

Concern	What to monitor or choose
RPM	Requests per minute; bursts can cause 429 responses
TPM	Tokens per minute; input plus reserved output budget
PTU	Reserved throughput for predictable high-volume workloads
Concurrent jobs	Service-specific limits, such as video generation
Regional capacity	Availability and failover options
Token cost	Input, output, cached, and tool-related tokens

Optimization levers:

- Set the smallest useful output limit
- Stream long responses where appropriate
- Cache stable prompts and retrieval results
- Use a smaller model for classification or routing
- Batch offline work when latency is not important
- Spread load across deployments or regions
- Establish a baseline before changing the model or prompt

Retrieval Practice — Lab 8

Pause here and complete Lab 8 before continuing.

glasspaper oblicum

CI/CD and Environment Promotion

[EXAM]

Essential: treat prompts, agent definitions, tool schemas, and evaluation datasets as versioned application assets.

- Use separate development, test, and production deployments
- Run quality and safety evaluations before promotion
- Keep endpoints, keys, resource IDs, and deployment names in secure environment configuration
- Monitor releases and retain a rollback path

Retrieval Practice — Lab 9

Pause here and complete Lab 9, then check its solution.

glasspaper oblicum

Observability and Evaluation in Production

[EXAM]

Collect signals at three levels:

Level	Signals
Application	Requests, failures, dependencies, user feedback
Model	Tokens, latency, throughput, model version, rate limits
AI quality	Groundedness, relevance, coherence, fluency, safety, refusal rate

Tracing should connect: user request → prompt → model response → retrieval → tool calls → final answer.

Drift means behavior or data changes over time:

- Input distribution changes
- Retrieval relevance declines
- Model version changes
- Groundedness or safety scores regress
- User feedback changes

Use evaluation datasets and alerts to detect regression before changing production behavior.

Retrieval Practice — Lab 10

Pause here and complete Lab 10, then check its solution.

Audit, Provenance, and Oversight

[EXAM]

Audit record: who requested the action, which model and prompt version ran, which tools and data were used, what the model returned, and which human approved it.

Provenance: retain citations, document IDs, source timestamps, model version, and evaluation version so an answer can be explained later.

Approval controls:

- Require approval before high-impact or irreversible tool calls
- Restrict tools by agent, user, environment, and task
- Escalate low-confidence decisions to a person
- Log both the requested action and the approved action
- Support block, rollback, and incident-response procedures

Exam takeaway: autonomy must be bounded by identity, permissions, tools, monitoring, and human oversight.

Key Takeaways

1. **Identity:** prefer Entra ID and managed identity; keyless still requires RBAC
2. **Network:** private endpoints and network rules control reachability
3. **Capacity:** RPM, TPM, PTU, concurrency, region, and token cost are separate concerns
4. **Delivery:** version AI configuration and promote it through tested environments
5. **Monitoring:** combine application, model, and AI-quality signals
6. **Governance:** preserve provenance and require approval for high-impact actions
7. **MSLearn relationship:** authentication, cost, tracing, and approvals appear indirectly; private networking and the full operational model are exam-oriented gaps

glasspaper oblicum

Appendix

Relationship to the Microsoft Learn path

Coverage Map

This topic supplements the four core presentations. It does not replace the official Microsoft Learn modules.

Status	Meaning
Direct	A dedicated or detailed Microsoft Learn lesson exists
Indirect	Microsoft Learn mentions the concept, but does not teach the full objective
Exam gap	Explicit in the exam objectives, but not developed in the scraped course

Exam objective	MSLearn status
Managed identity and keyless authentication	**Indirect** — appears in authentication and agent lessons
Private networking and role policies	**Exam gap** — not developed as an architecture topic
Quotas, scaling, and cost footprints	**Indirect** — model cost, latency, and quota examples exist
CI/CD for Foundry projects	**Indirect** — discussed in Microsoft 365 Toolkit material
Monitoring, tracing, drift, and grounding quality	**Indirect** — evaluation and tracing are introduced, operations are lighter
Audit, provenance, approval, and oversight	**Indirect** — responsible AI and workflow approval are covered separately